

Podstawy Uczenia Maszynowego

Czym się zajmiemy na ćwiczeniach?

~~Ćwiczenia 1: metody liniowe~~

~~Ćwiczenia 2: metody oparte na sąsiedztwie~~

~~Ćwiczenia 3: metody jądrowe~~

~~Ćwiczenia 4: metody probabilistyczne~~

~~Ćwiczenia 5: klasyfikacja niezbalansowana~~

Ćwiczenia 6: selekcja cech i redukcja wymiarowości

Ćwiczenia 7: klasteryzacja

Selekcja cech i redukcja wymiarowości

Motywacja

- cechy mają różną **jakość (quality)** oraz **siłę predykcyjną (predictive power)**
- potencjalne problemy:
 - szum
 - outliery
 - redundantne informacje
 - brak znaczenia dla problemu
- chcemy używać **jak najmniej cech**: tańsze, szybsze, wygodniejsze, redukuje powyższe problemy

Motywacja

- dane mają pewną **wymiarowość (dimensionality)**
- **idea:**
 - ta wymiarowość jest za duża, mamy za dużo informacji
 - niepotrzebne wymiary - szum, outliery, słaba korelacja z targetem etc.
- interesuje nas **istotna wymiarowość (intristic dimension)**
- podprzestrzeń realnie ważna dla problemu
- typowo (dużo) mniejsza od wejściowej

Selekcja cech (feature selection)

- wybieramy **podzbiór cech (feature subset)**
- typowo oblicza **ważność cech (feature importance)** i usuwa te najmniej wartościowe
- może rozważać cechy z osobna albo kombinacje
- **zaleta:** interpretowalność
- **wada:** nie przekształca cech, nie rozważa złożonych kombinacji

Redukcja wymiarowości (dimensionality reduction)

- **rzutujemy cechy (feature projection)** na nową przestrzeń o mniejszej liczbie wymiarów
- tworzy nowe, złożone cechy
- **zaleta:** rozważa złożone przekształcenia całej przestrzeni cech
- **wada:** nieinterpretowalne

- **uwaga:** niektórzy uznają "dimensionality reduction" za ogólne pojęcie, które dzieli się na "feature selection" oraz "feature projection"

Zastosowania selekcji cech

- **ulepszanie wyników modeli:**
 - może usuwać szum i poprawiać wyniki klasyfikacji, regresji etc.
 - szczególnie ważne dla modeli, które używają cech wprost, bez przekształceń:
 - modele liniowe
 - kNN
 - Naive Bayes
 - szczególnie zysk dla cech zaszumionych, niskiej jakości, dziwnych, generowanych automatycznie etc.

Zastosowania selekcji cech

- **genetyka:**
 - sekwencjonowanie genów daje tysiące cech
 - z góry wiadomo, że większość z nich nie ma znaczenia dla problemu, np. konkretnej choroby
 - często celem nie jest model predykcyjny, tylko analiza, które geny są najslabiej / najmocniej związane z chorobą

Zastosowania selekcji cech

- **analitika biznesowa / business intelligence:**
 - trzeba zrozumieć zachowanie i zainteresowania klientów
 - identyfikacja najslabszych i najmocniejszych cech
 - np. precyzyjniejsze dobranie celów kampanii marketingowych
 - zastosowania i predykcyjne, i analityczne

Zastosowania redukcji wymiarowości

- **wizualizacja:**

- redukcja do 2-3 wymiarów
- pozwala zobrazować wysokowymiarowe dane
- np. analiza klastrow czy zależności w danych

- **ulepszanie wyników modeli:**

- kiedy mamy bardzo dużo cech, nieliniowe korelacje etc., to redukcja wymiaru jest elastyczna
- w tych wypadkach interpretowalność selekcji cech nas nie interesuje

Selekcja cech

Podejścia do selekcji cech

- istnieją **3 grupy method**:
 - filter
 - embedded
 - wrapper

Filter methods

- waga cechy jest obliczana na podstawie jakiejś ogólnej miary jakości cech
- typowo oparte o statystykę, teorię informacji itp.
- typowo metody sprawdzające **jedną zmienną naraz (univariate)**

Zalety:

- prostota
- szybkość, skalowalność
- działają dla małych danych
- dla dowolnych modeli

Wady:

- typowo nie uwzględnia interakcji cech
- nie bierze pod uwagę cech dalszych modeli
- typowo najgorsze wyniki

Embedded methods

- wagi cech są obliczane przez model ML jako część procedury treningu
- wymaga odpowiednich modeli, np. lasów losowych

Zalety:

- dość szybkie
- niezłe wyniki
- uwzględnia interakcje cech

Wady:

- mocno związane z typem modelu
- wymaga odpowiednich modeli
- działa dobrze wtedy, jeżeli sam model jest dobry

Wrapper methods

- algorytmy wykorzystujące modele ML jako funkcję w środku do obliczania wartości cech
- "wrapują" model ML w dodatkowe procedury

Zalety:

- typowo najlepsze wyniki
- uwzględnia złożone interakcje cech

Wady:

- koszt obliczeniowy
- działa dobrze wtedy, jeżeli sam model jest dobry
- podatne na overfitting

Selekcja cech - filter methods

Próg wariancji (variance threshold)

- obliczamy **wariancję (variance)** każdej cechy i usuwamy te o zbyt niskiej wariancji
- takie cechy są zbyt mało zmienne, żeby być informatywne
- **wymaga przeskalowania cech** - wariancja zależy od skali!
- typowo niski próg, np. 0.01

Zalety:

- prostota
- eliminuje oczywiste słabe cechy
- nienadzorowane

Wady:

- zbyt agresywne dla zmiennych rzadkich

Korelacja cech

- sprawdzamy **liniową korelację (linear correlation)** między cechami
- eliminujemy najmocniej skorelowane cechy - zawierają redundantne informacje
- usuwa się według wartości bezwzględnej, np. z progiem 0.9
- **problem:**
 - jak mamy parę skorelowanych cech, to którą usunąć?
 - losowo
 - tę o mniejszej wariancji
 - tę, która ma średnio większe korelacje z innymi cechami

Korelacja cech - zalety i wady

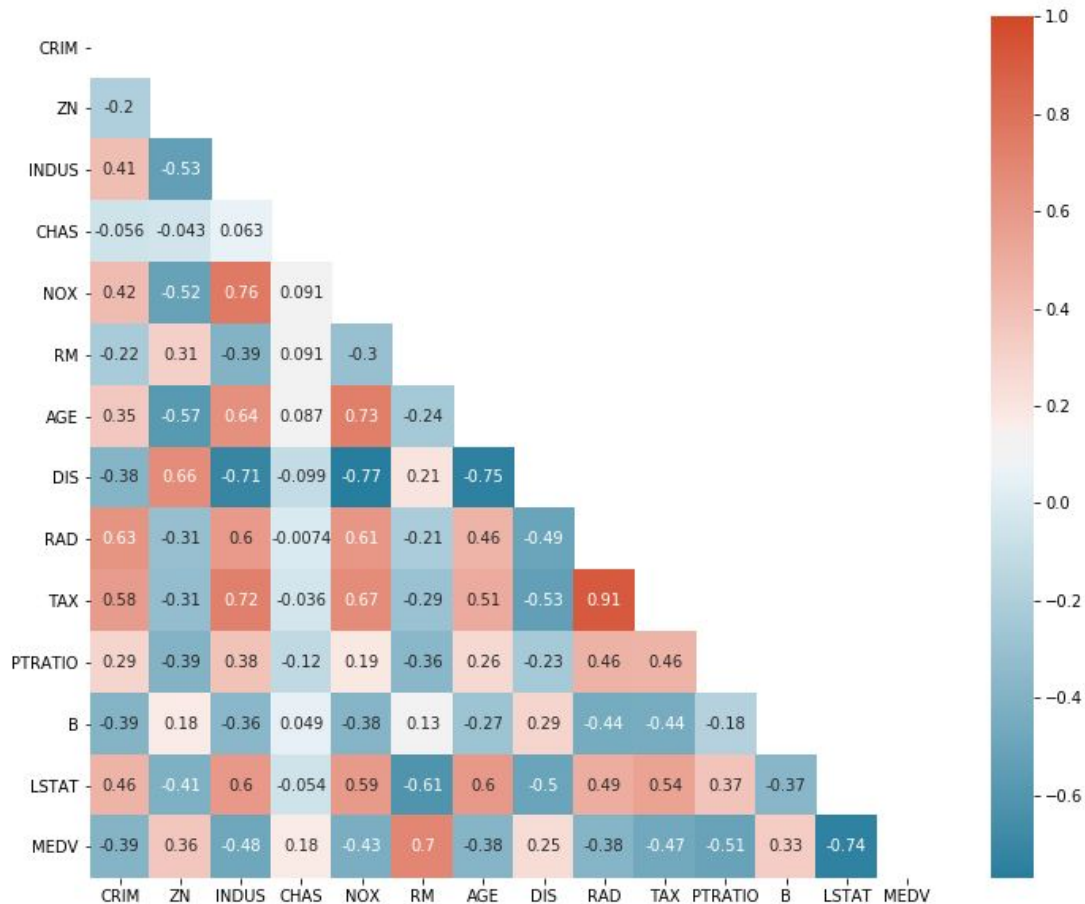
Zalety:

- prostota
- eliminuje oczywiste redundancje
- nienadzorowane

Wady:

- nie usuwa nieliniowych korelacji (to nie zawsze wada!)
- uwzględnia tylko interakcje par cech

Feature correlation heatmap



Korelacja ze zmienną zależną

- obliczamy **korelację** między cechą a zmienną zależną
- na zbiorze treningowym
- **tylko do regresji** - wymaga dwóch wektorów ciągłych
- wysoka korelacja = cecha liniowo związana z tym, co chcemy przewidywać

Zalety:

- prostota
- interpretowalność
- dobrze identyfikuje mocne cechy

Wady:

- nie uwzględnia nieliniowych korelacji
- nie uwzględnia redundancji cech

Mutual Information

- **informacja wzajemna (mutual information)** między zmiennymi losowymi x oraz y
- miara **ilości informacji** (w sensie Shannona), jak dużo zaobserwowanie jednej zmiennej mówi nam o możliwych wartościach drugiej zmiennej
- dla zmiennych ciągłych:

$$I(X; Y) = \int_{y \in Y} \int_{x \in X} P(x, y) \log \left(\frac{P(x, y)}{P(x)P(y)} \right) dx dy$$

- **obserwacja:** gdy zmienne są niezależne (jedna nic nie mówi o drugiej), to MI to zero:

$$P(x, y) = P(x) * P(y) \rightarrow \log(1) = 0 \rightarrow I(X; Y) = 0$$

Mutual Information

- **problem:** jak obliczyć taką całkę dla dowolnych rozkładów?

$$I(X; Y) = \int_{y \in Y} \int_{x \in X} P(x, y) \log \left(\frac{P(x, y)}{P(x)P(y)} \right) dx dy$$

- **rozwiązanie:** estymacja z pomocą metod najbliższych sąsiadów
- **szybkie**, bo mamy 1-wymiarowe rozkłady: x to cecha, y to zmienna zależna
- jest dużo algorytmów do estymacji MI, to istotny problem w statystyce
- ma wersje do regresji i do klasyfikacji
- **modeluje zależności nieliniowe** - nieparametryczne, nie zakłada nic co do rozkładów

Mutual Information - zalety i wady

Zalety:

- bardzo dobre wyniki
- intuicyjna definicja
- mocne podstawy teoretyczne
- modeluje zależności nieliniowe
- działa dla cech rzadkich

Wady:

- ma hiperparametr k (liczba najbliższych sąsiadów), ale typowo ma mały wpływ
- obliczanie jest nietrywialne i trudne w zrozumieniu

Mutual Information - artykuły

- [StatQuest - "Mutual Information, Clearly Explained"](#)
- [CrossValidated - mutual information vs correlation](#)
- przegląd różnych algorytmów do estymacji MI ([link](#)):

Schaffernicht, Erik, et al. *"On estimating mutual information for feature selection."* International Conference on Artificial Neural Networks, 2010.

- wersja dla regresji ([link](#)):

Kraskov, Alexander, Harald Stögbauer, and Peter Grassberger. *"Estimating mutual information."* Physical Review E 69.6 (2004): 066138.

- wersja dla klasyfikacji ([link](#)):

Ross, Brian C. *"Mutual information between discrete and continuous data sets."* PLOS One 9.2 (2014): e87357.

Selekcja cech - embedded methods

Modele liniowe

- model liniowy przypisuje **wagę do każdego wymiaru**
- wymaga znormalizowanych cech
- waga = **miara, jak mocno cecha wpływa na predykcje modelu**
- wybieramy cechy o największej wartości bezwzględnej wagi
- typowo używa się modeli z regularyzacją, np. ridge, LASSO
- **heurystyka:** nie mierzy jakości cech jako takich, tylko siłę ich wpływu na model
- **problem:** przy mocno skorelowanych cechach ważność "dzieli się" pomiędzy nie

Modele liniowe - zalety i wady

Zalety:

- prostota
- szybkość
- skalowalność
- interpretowalność

Wady:

- słabo uwzględnia interakcje cech
- mocno heurystyczne
- mocno zależne od jakości samego modelu liniowego, które są często zbyt proste

Modele drzewiaste

- drzewo decyzyjne przy treningu wybiera **punkt podziału** - cechę i wartość
- wybiera się punkt **minimalizujący koszt po podziale** - chcemy jak najlepiej odróżnić klasy od siebie
- w lesie losowym lub boostingu każda cecha zostanie **wielokrotnie wybrana**
- **idea:** dobre cechy to takie, które typowo dobrze oddzielają klasy od siebie
- waga cechy to **średnia redukcja kosztu (Mean Decrease in Impurity, MDI)** w ensemble'u drzewiastym dla tych podziałów, które używają danej cechy
- koszt to typowo **entropia** - dobre podstawy teoretyczne

Modele drzewiaste - zalety i wady

Zalety:

- uwzględnia złożone interakcje cech
- dobre podstawy teoretyczne
- typowo dają dobre wyniki, a więc też oszacowanie ważności cech

Wady:

- preferuje zmienne numeryczne i o dużej liczbie unikalnych wartości
- słabo działa dla cech rzadkich
- zależne od losowości - wymaga wielu drzew, żeby dostać stabilny wynik

Selekcja cech - wrapper methods

Recursive Feature Elimination (RFE)

- **idea:** użyć modelu do mierzenia jakości cech, ale wielokrotnie
- eliminuje przede wszystkim redundantne cechy i szum
- **algorytm:**
 - wytrenuj model ML (np. las losowy)
 - usuń naj słabszą cechę (według oceny modelu)
 - powtarzaj, aż uzyskasz pożądaną liczbę cech
- istnieje też wariant **z walidacją skośną** - **RFECV**, w którym zostawiamy cechy, dla których osiągnięto największą walidację skośną

RFE - zalety i wady

Zalety:

- prostota
- nieźle eliminuje redundantne cechy
- może wybrać automatycznie liczbę cech do eliminacji

Wady:

- koszt obliczeniowy (szczególnie RFECV)
- wymaga dobrych modeli, żeby dobrze działać
- podatne na overfitting

Redukcja wymiarowości

Podejścia do redukcji wymiarowości

- możemy to zrobić **liniowo** albo **nieliniowo**
- różnica tkwi w tym, **jak rzutujemy** na niższy wymiar:
 - liniowo - rzutowanie to **przekształcenie liniowe**, typowo mnożenie macierzowe
 - nieliniowo - rzutowanie to **funkcja nieliniowa**, często nieparametryczna
- różnice są typowe:
 - liniowo: szybkie, proste, bardziej interpretowalne, ale uwzględnia tylko kombinacje liniowe wag i cech
 - nieliniowo: potrafi odwzorować zależności nieliniowe, ale trudniejsze i wolniejsze

Podejścia liniowe

- **Principal Component Analysis (PCA)**
- **Singular Value Decomposition (SVD)**
- Sparse PCA
- Robust PCA
- Nonnegative Matrix Factorization (NMF)
- Multiple Correspondence Analysis (MCA)
- Factor Analysis of Mixed Data (FAMD)

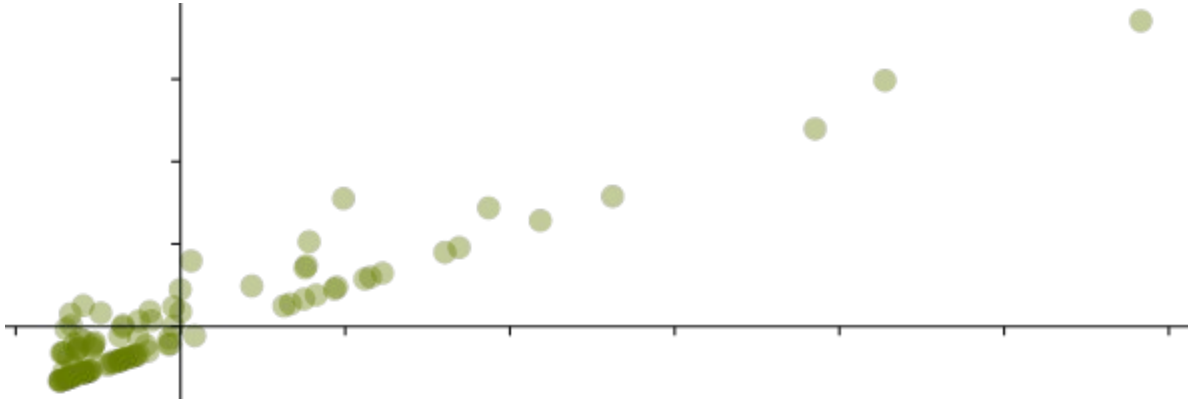
Podejścia nieliniowe

- **Uniform Manifold Approximation and Projection (UMAP)**
- **t-distributed Stochastic Neighbor Embedding (t-SNE)**
- kernel PCA
- autoenkodery
- Isomap
- Locally Linear Embedding (LLE)

Principal Component Analysis (PCA)

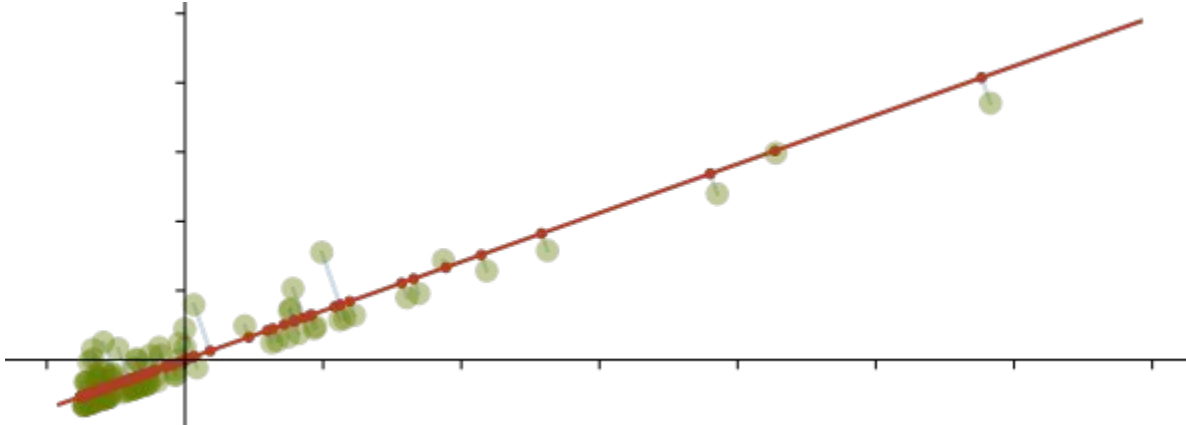
PCA - idea wizualnie

- mamy 2 cechy, chcemy je zredukować do jednej
- jaka cecha byłaby tutaj dobra?



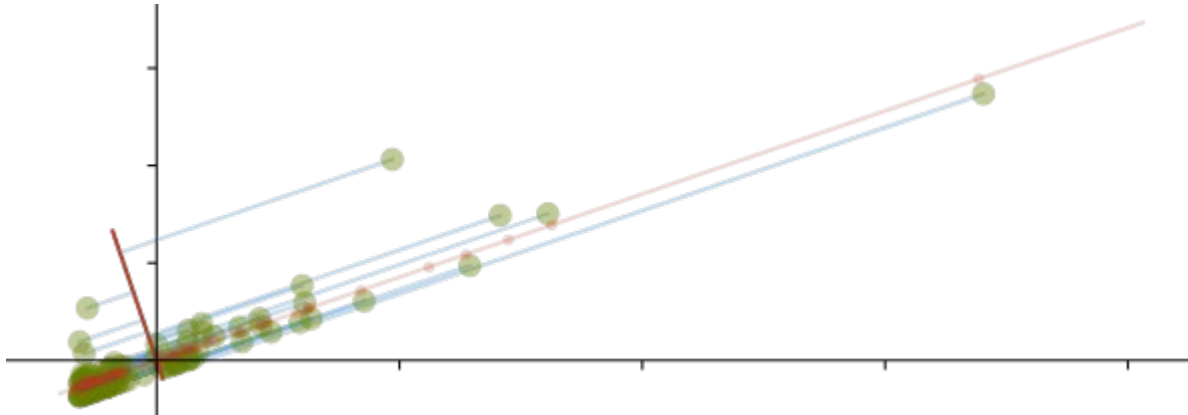
PCA - idea wizualnie

- **maksymalizujemy wariancję** - "łapiemy" jak najwięcej zmienności cechy
- **minimalizujemy błąd** - jesteśmy "blisko" oryginalnych punktów
- ta cecha to **principal component (PC)**



PCA - idea wizualnie

- możemy dołożyć drugi PC, **ortogonalny** do pierwszego
- ma większy błąd i wykrywa mniej wariancji, więc jest **ogólnie mniej ważny**



PCA - centrowanie

- PCA zakłada, że cechy są **wycentrowane**
- jeżeli nasze cechy mają różną skalę, to dodatkowo je **standaryzujemy**
- powyższe gwarantuje, że cechy mają równą wagę
- **wada:** nie działa dla cech rzadkich, centrowanie niszczy rzadkość
- dla cech rzadkich są inne algorytmy, np. SVD

PCA - algorytm

- znajdujemy **wektor (kombinację cech)**, która **minimalizuje błąd**
- błąd = suma kwadratów odległości między punktami a tym wektorem (SSE)
- dla uproszczenia (baza kanoniczna) zakładamy, że jest **jednostkowy** (długość 1)
- odpowiada to wektorowi o **najwyższej wariancji**
- jest to **pierwszy principal component**
- potem znajdujemy kolejny, **ortogonalny** względem poprzedniego, maksymalizując pozostałą wariancję (to, czego nie "złapał" pierwszy)
- powtarzamy po kolei, aż do osiągnięcia pożądanej liczby PC

Principal components

- każdy PC to wektor, oznaczający **wagi cech**
- jak dużo cecha "kontrybuuje" do danego PC, jak ważna dla niego jest
- jeżeli zbierzemy je jako kolumny w macierzy W , to mamy:
$$X' = XW$$
- redukcja wymiaru to **rzut liniowy**, czyli po prostu mnożenie przez macierz
- **problem:** jak policzyć te principal components?

PCA i macierz kowariancji

- **chcemy wektorów:**
 - łapiących najwięcej wariancji
 - nieskorelowanych = o zerowej kowariancji
- takie informacje zapewnia **macierz kowariancji:**

$$Cov(X) = \frac{X^T X}{n - 1}$$

- **uwaga:** powyższy wzór zakłada wycentrowane dane!
- **problem:** jak wyciągnąć te szczególnie ciekawe wektory z tej macierzy?

PCA i eigendecomposition

- **rozkład według wartości własnych (eigendecomposition)** macierzy kowariancji:

$$Cov(X) = V L V^T$$

- V w kolumnach ma **wektory własne (eigenvectors)**
- L to macierz przekątniowa z malejącymi **wartościami własnymi (eigenvalues)**
- **formalnie:** wektory własne nie zmieniają kierunku po transformacji liniowej, a tylko są skalowane przez wartość własną
- **geometrycznie:**
 - eigenvector to kierunek, w którym przestrzeń się "rozciąga"
 - eigenvalue to "siła" takiego rozciągania
- **wniosek:** w macierzy kowariancji "największe rozciąganie" to taki wektor, który ma największą wariancję!

PCA i eigendecomposition

- **principal components to eigenvectors** macierzy kowariancji
- eigenvectors są **ortonormalne** - idealna baza przestrzeni!
- mamy więc: $X' = XV$
- **problem:** złożoność obliczeniowa to $O(n^3)$, mnożenie macierzy i eigendecomposition

$$Cov(X) = \frac{X^T X}{n - 1} = V L V^T$$

- **idea:** znajdziemy narzędzie, które da nam to samo, tylko szybciej

SVD i PCA

- PCA można obliczyć **używając SVD**
- SVD: $X = U\Sigma V^T$
- **uwaga:** U i V^T są ortogonalne, tzn. $U^{-1} = U^T$
- PCA:

$$Cov(X) = \frac{X^T X}{n-1} = \frac{(U\Sigma V^T)^T U\Sigma V^T}{n-1} = \frac{V\Sigma U^T U\Sigma V^T}{n-1} = V \frac{\Sigma^2}{n-1} V^T$$

$$Cov(X) = V L V^T$$

Po co nam PCA?

- PCA to takie "przeskalowane" SVD, w sumie niewiele się różni
- no to po co?
- **interpretacja** - maksymalizacja wariancji, "główne kierunki" w danych
- **optymalna liniowa redukcja wymiarowości** - najmniejszy MSE z metod liniowych
- **rozwinienia** - pełne sformułowanie statystyczne umożliwia dużo rzeczy:
 - regularyzację L1 (Sparse PCA), L2 (Ridge PCA)
 - sformułowanie probabilistyczne (Probabilistic PCA), estymatę MLE
 - wersje nieliniowe (Kernel PCA)

Interpretacja PCA

- zrozumienie, **co znaczą principal components** bywa trudne
- można sprawdzić tzw. **loadings**, czyli jak dużą wagę w danym PC mają wejściowe cechy
- wzór:

$$\text{loadings} = V \frac{\Sigma}{n - 1}$$

- ciężko powiedzieć, co to znaczy "dużo" cechy w danym PC
- **uwaga:** jest tutaj pewien problem nazewniczy (patrz [tu](#), [tu](#) i [tu](#)), ale powyższa definicja jest formalnie poprawna

PCA i macierz korelacji

- PCA definiuje się typowo na **macierzy kowariancji**
- **problem:** kowariancja jest czuła na skalę cech
- jeżeli **ustandaryzujemy** cechy, to jest ona równa **macierzy korelacji**
- robimy zawsze, kiedy nasze cechy mają różną skalę (czyli często)

PCA - złożoność obliczeniowa

- **tak samo, jak SVD**
- trening: $O(nd^2)$
- minimalnie większa stała
- ma też wersję z minibatchami dla naprawdę dużych danych

PCA - zalety i wady

Zalety:

- szybkość
- stabilność numeryczna
- dobre dla danych gęstych
- daje nieskorelowane, ustandaryzowane cechy
- intuicyjna interpretacja
- usuwa szum z danych (niepotrzebną wariancję)
- mocne podstawy statystyczne

Wady:

- PC są trudne w interpretacji
- może pogorszyć wyniki, bo usuwa część informacji
- nie działa dla danych rzadkich

PCA - źródła

- [A friendly introduction to Principal Component Analysis - P. Bloem](#)
- [Relationship between SVD and PCA. How to use SVD to perform PCA?](#)

Do dzieła!

Slajdy bonusowe

Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD)

$$X = U \Sigma V^T \quad \begin{bmatrix} | & | & & | \\ u_1 & u_2 & \cdots & \\ | & | & & | \end{bmatrix} \quad \begin{bmatrix} \sigma_1 & 0 & \cdots \\ 0 & \sigma_2 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \quad \begin{bmatrix} - & v_1 & - \\ - & v_2 & - \\ & \vdots & \end{bmatrix}$$

- X - dowolna macierz rzeczywista, kształtu $n \times d$
- U, V^T - **macierze ortogonalne** kształtu: U - $n \times n$, V^T - $d \times d$
- Σ - macierz przekątniowa z wartościami posortowanymi malejąco, kształtu $n \times d$

Macierze ortogonalne

- **własności:**
 - kolumny mają normę jednostkową (długość 1)
 - kolumny są wzajemnie ortogonalne
- **wnioski:**
 - taka macierz definiuje **kanoniczną bazę przestrzeni (canonical basis)**
 - jest to **świetny** zestaw cech do ML, bo nie ma żadnych korelacji cech!

SVD - wizualizacja

- tylko **d pierwszych** kolumn U ma znaczenie - pozostałe są sztuczne, dla formalności
- w praktyce się tego nie liczy, mamy tzw. **economy SVD**

$$\begin{array}{c} \text{Full SVD} \\ \left[\begin{array}{c} \text{X} \end{array} \right] = \underbrace{\left[\begin{array}{c|c} \hat{\text{U}} & \hat{\text{U}}^\perp \end{array} \right]}_{\text{U}} \underbrace{\left[\begin{array}{c} \hat{\Sigma} \\ \hline 0 \end{array} \right]}_{\Sigma} \left[\begin{array}{c} \text{V}^* \end{array} \right] \\ \text{Economy SVD} \\ \left[\begin{array}{c} \text{X} \end{array} \right] = \left[\begin{array}{c} \hat{\text{U}} \end{array} \right] \left[\begin{array}{c} \hat{\Sigma} \\ \hline \end{array} \right] \left[\begin{array}{c} \text{V}^* \end{array} \right] \end{array}$$

Truncated SVD

- **ucinamy** ostatnie wiersze/kolumny, zostawiając te dla największych wartości z macierzy Σ
- odrzucamy "najmniej ważne" części przestrzeni, zachowując "istotę problemu"

$$\begin{array}{c}
 \text{Full SVD} \\
 \left[\begin{array}{c} X \end{array} \right] = \underbrace{\left[\begin{array}{c|c|c} \tilde{U} & \hat{U}_{\text{rem}} & \hat{U}^\perp \end{array} \right]}_{\tilde{U}} \left[\begin{array}{c|c} \begin{array}{c} \tilde{\Sigma} \\ \hline \hat{\Sigma}_{\text{rem}} \\ \hline 0 \end{array} & \begin{array}{c} \tilde{V}^* \\ \hline V_{\text{rem}} \end{array} \end{array} \right] \\
 \\
 \text{Truncated SVD} \\
 \approx \left[\begin{array}{c} \tilde{U} \end{array} \right] \left[\begin{array}{c} \tilde{\Sigma} \\ \hline \end{array} \right] \left[\begin{array}{c} \tilde{V}^* \end{array} \right]
 \end{array}$$

SVD i redukcja wymiarowości

- **idea:** cechy są ułożone w kolejności "ważności" - wielkości elementów w Σ
- macierz V' to nasza nowa **baza przestrzeni**
- **trening:** oblicz economy SVD, zostaw tylko pierwszych d' kolumn z V
- **transformacja:** zrzuć X na niższy wymiar: $X' = XV'$
- **dla cech rzadkich** - SVD ma do tego wydajne implementacje
- **heurystyka** - nie ma głębokiej interpretacji, po prostu "kompresuje" przestrzeń

SVD - złożoność obliczeniowa

- **złożoność treningu:** $O(nd^2)$
- **złożoność transformacji:** $O(ndd')$
- transformacja - mnożenie dwóch macierzy prostokątnych $X_{n \times d}$ oraz $V_{d \times d'}$

SVD - zalety i wady

Zalety:

- szybkość
- działa dla danych rzadkich
- bardzo stabilne numerycznie

Wady:

- trudne w interpretacji
- heurystyczne, nie dostosowane konkretnie do redukcji wymiarowości
- czułe na skalę cech - wymaga normalizacji

SVD - źródła

- [Explained: Singular Value Decomposition \(SVD\) - E. Storrs](#)
- [Singular Value Decomposition playlist - Steve Brunton \(YT\)](#)
- ["Data Driven Science & Engineering" S. Brunton, J. Kutz](#)
- [High Dimensional Data Analysis \(HDDA\) course - SVD](#)
- [Stanford CS168: The Modern Algorithmic Toolbox Lecture #9: The Singular Value Decomposition \(SVD\) and Low-Rank Matrix Approximations](#)

Permutation Feature Importance

Permutation Feature Importance

- **idea:** jeżeli usuniemy dobrą cechę, to wyniki modelu powinny mocno spaść
- **algorytm:**
 - wytrenuj model ML, sprawdź wynik na zbiorze walidacyjnym, zapisz wynik
 - idź po kolei po cechach, dla każdej powtórz wielokrotnie:
 - przemieszaj losowo wartości cechy na zbiorze walidacyjnym
 - sprawdź i zapisz wynik na zbiorze walidacyjnym
 - oblicz średni wynik modelu dla przemieszanej cechy
 - waga cechy to różnica między wynikiem oryginalnym i po przemieszaniu

Permutation Feature Importance

- uwzględnia **interakcje cech**, bo przemieszanie wartości cechy niszczy też relacje między cechami
- nie wymaga retrenowania modelu, patrzymy tylko na zbiór walidacyjny
- istnieją **wersje alternatywne**:
 - użycie zbioru treningowego - bardziej analiza modelu niż obiektywnej ważności cech
 - użycie zbioru testowego - tylko, gdy interesuje nas analiza modelu post-hoc
 - użycie walidacji skróśnej - większy koszt, ale lepsze oszacowanie

Permutation Feature Importance - zalety i wady

Zalety:

- intuicyjne, interpretowalne
- niski koszt obliczeniowy
- uwzględnia interakcje między cechami
- pozwala na analizę gotowego modelu

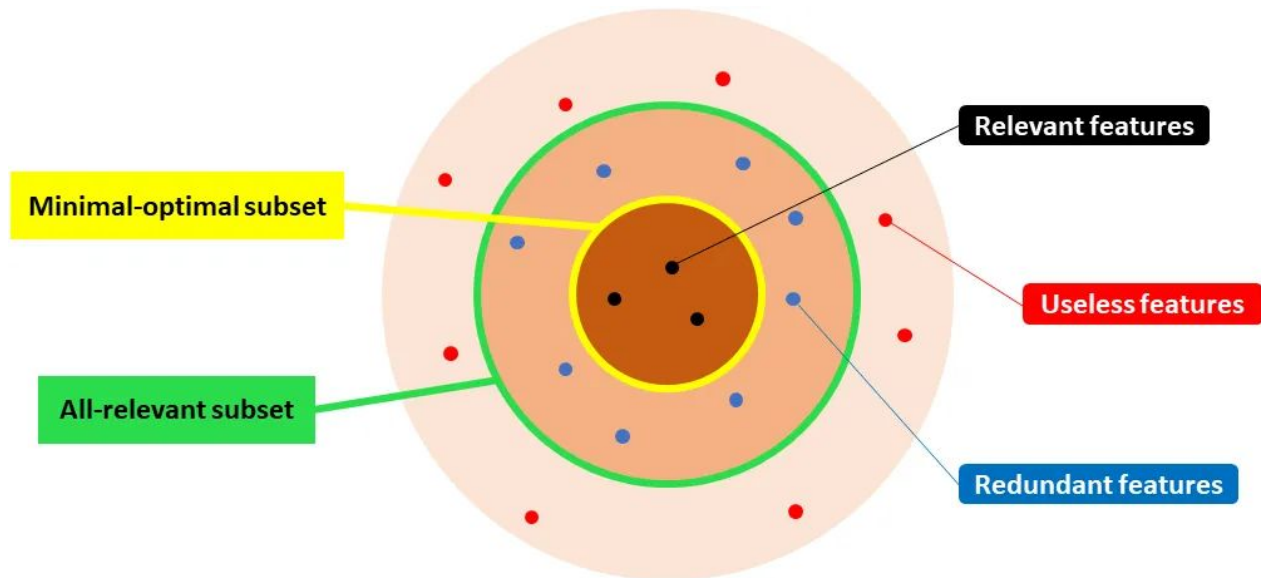
Wady:

- wymaga zbioru walidacyjnego
- bardzo mocno związane z modelem

**All relevant
feature selection**

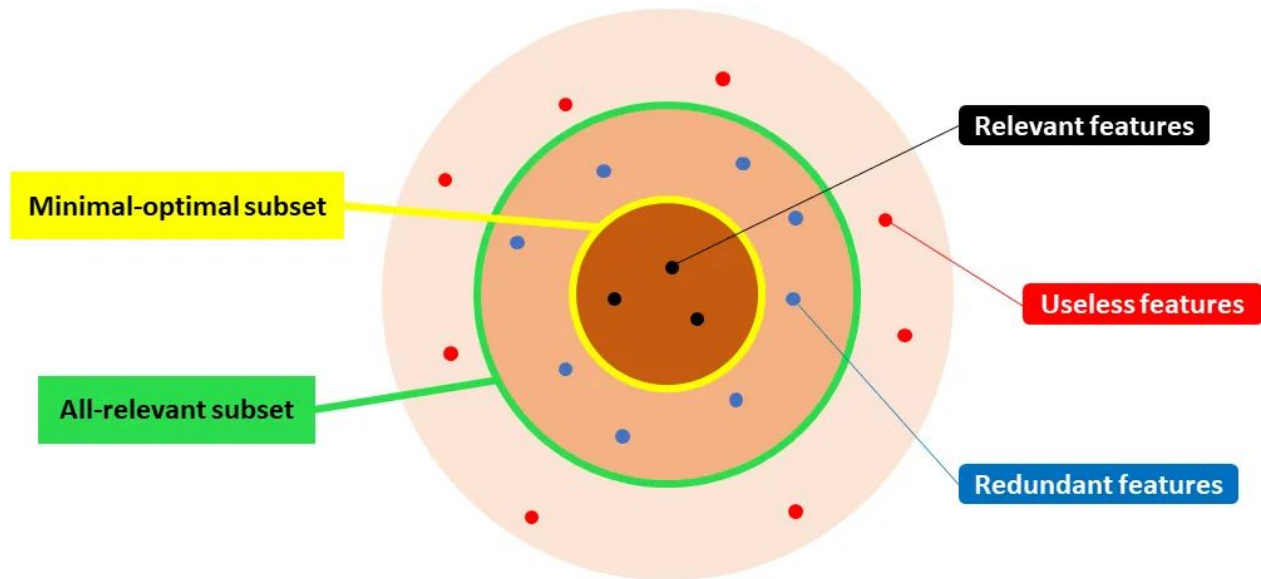
Podejścia do selekcji cech

- **minimal-optimal feature selection (MOFS)** próbuje znaleźć minimalny podzbiór cech o maksymalnej mocy predykcyjnej
- wszystkie dotychczas przedstawione metody
- najpopularniejsze, skupione mocno na ulepszeniu wyniku predykcji danym modelem



Podejścia do selekcji cech

- **all relevant feature selection (ARFS)** próbuje znaleźć wszystkie te cechy, które mają dobrą siłę predykcyjną dla problemu
- ważny w zastosowaniach analitycznych i tam, gdzie cechy są mocno niezależne
- np. bioinformatyka, genetyka



Boruta algorithm

- **algorytm Boruty (Boruta's algorithm / Boruta algorithm)** rozwiązuje problem all-relevant feature selection

Kursa, Miron B., Aleksander Jankowski, Witold R. Rudnicki. *"Boruta – a system for feature selection."* Fundamenta Informaticae 101.4 (2010): 271-285.

- **dla dowolnych modeli** - wybierze zestaw ogólnie sensownych cech, które można potem dalej filtrować pod konkretny model
- **mało podatny na overfitting** - ogólna własność podejścia ARFS
- selekcja typu wrapper (ale do ARFS)

Boruta algorithm - shadow features

- najpierw tworzymy **shadow features**, czyli przemieszane losowo oryginalne cechy
- doklejamy je do zbioru - mamy 2 razy więcej cech, z czego połowę losową

	age	height	weight	shadow_age	shadow_height	shadow_weight
0	25	182	75	51	176	75
1	32	176	71	32	182	71
2	47	174	78	47	168	78
3	51	168	72	25	181	72
4	62	181	86	62	174	86

Boruta algorithm - hits

- trenujemy Random Forest na podwójnych cechach, sprawdzamy feature importances
- **idea:** cecha jest ważna tylko wtedy, jeżeli jest ważniejsza od losowych cech
- **importance threshold** = najwyższa waga cechy spośród shadow features
- oryginalny atrybut zostaje uznany za ważny - **hit** - jeżeli ma ważność wyższą od progu

Boruta algorithm - powtórzenia

- **powtarzamy** poprzednie kroki wiele razy (np. 20-30):
 - stworzenie losowych permutowanych cech - shadow features
 - trening Random Forest
 - zliczenie hits (czy cecha jest ważniejsza od "najlepszej" shadow feature)
- mamy **rozkład dwumianowy** dla każdej cechy:
 - seria 0-1: czy cecha była ważniejsza od shadow features, czy nie
 - możemy wykonać **test statystyczny**

Boruta algorithm - test statystyczny

- **hipoteza zerowa:** ważność cechy jest równa ważności najważniejszej shadow feature
- **rodzaj testu:** obustronny test równości (two-tailed equality test)
- **poziom istotności α :** typowo 0.05
- stosujemy **korektę Bonferroniego**, bo mamy wielokrotne testowanie (wiele cech)
- odrzucamy hipotezę zerową w 2 przypadkach (bo mamy two-tailed test):
 - cecha mało ważna - usuwamy
 - cecha ważna - na pewno warto zostawić
- **usuwamy słabe cechy** i powtarzamy, aż oznaczymy wszystkie cechy

Rozwinięcia algorytmu Boruty

- [BorutaPy](#):
 - luźniejsza wersja, wybiera więcej cech
 - cecha ma być ważniejsza od percentyla shadow features, zamiast od najważniejszej
 - Benjamini-Hochberg FDR zamiast korekty Bonferroniego
- [Leshy](#):
 - LightGBM zamiast Random Forest
 - szybsze + działa dla cech kategorycznych

Boruta algorithm - zalety i wady

Zalety:

- mocne podstawy statystyczne
- bardzo dobrze wykrywa słabe cechy
- koryguje problemy Random Forest

Wady:

- koszt obliczeniowy
- jak chcemy po prostu selekcji cech do treningu, to trzeba dodatkowej selekcji